

- Cody, W.J. 1988, "MACHAR: A Subroutine to Dynamically Determine Machine Parameters," *ACM Transactions on Mathematical Software*, vol. 14, pp. 303–311.[3]
- Malcolm, M.A. 1972, "Algorithms to Reveal Properties of Floating-Point Arithmetic," *Communications of the ACM*, vol. 15, pp. 949–951.[4]
- Numerical Recipes Software 2007, "Code for Machar," *Numerical Recipes Webnote No. 27*, at <http://www.nr.com/webnotes?27> [5]
- Goldberg, D. 1991, "What Every Computer Scientist Should Know About Floating-Point Arithmetic," *ACM Computing Surveys*, vol. 23, pp. 5–48.

22.3 Gray Codes

A Gray code is a function $G(i)$ of the integers i that for each integer $N \geq 0$ is one-to-one for $0 \leq i \leq 2^N - 1$, and that has the following remarkable property: The binary representations of $G(i)$ and $G(i + 1)$ differ in *exactly one bit*. An example of a Gray code (in fact, the most commonly used one) is the sequence 0000, 0001, 0011, 0010, 0110, 0111, 0101, 0100, 1100, 1101, 1111, 1110, 1010, 1011, 1001, and 1000, for $i = 0, \dots, 15$. The algorithm for generating this code is simply to form the bitwise exclusive-or (XOR) of i with $i/2$ (integer part). Think about how the carries work when you add one to a number in binary, and you will be able to see why this works. You will also see that $G(i)$ and $G(i + 1)$ differ in the bit position of the rightmost zero bit of i (prefixing a leading zero if necessary).

The spelling is "Gray," not "grey": The codes are named after one Frank Gray, who first patented the idea for use in shaft encoders. A shaft encoder is a wheel with concentric coded stripes, each of which is "read" by a fixed optical sensor or conducting brush. The idea is to generate a binary code describing the angle of the wheel. The obvious, but wrong, way to build a shaft encoder is to have one stripe (the innermost, say) present on half the wheel, but absent on the other half; the next stripe is present in quadrants 1 and 3; the next stripe is present in octants 1, 3, 5, and 7; and so on. The optical or electrical sensors together then read a direct binary code for the position of the wheel.

The reason this method is bad is that there is no way to guarantee that all the brushes will make or break contact *exactly* simultaneously as the wheel turns. Going from position 7 (0111) to 8 (1000), one might pass spuriously and transiently through 6 (0110), 14 (1110), and 10 (1010), as the different brushes make or break contact. Use of a Gray code on the encoding stripes guarantees that there is no transient state between 7 (0100 in the sequence above) and 8 (1100).

Of course we then need circuitry, or algorithmics, to translate from $G(i)$ to i . Figure 22.3.1(b) shows how this is done by a cascade of XOR gates. The idea is that each output bit should be the XOR of all more significant input bits. To do N bits of Gray code inversion requires $N - 1$ steps (or gate delays) in the circuit. (Nevertheless, this is typically very fast in circuitry.) In a register with word-wide binary operations, we don't have to do N consecutive operations, but only $\ln_2 N$. The trick is to use the associativity of XOR and group the operations hierarchically. This involves sequential right-shifts by 1, 2, 4, 8, $\dots$ bits until the wordlength is exhausted. Here is a piece of code for doing both $G(i)$ and its inverse:

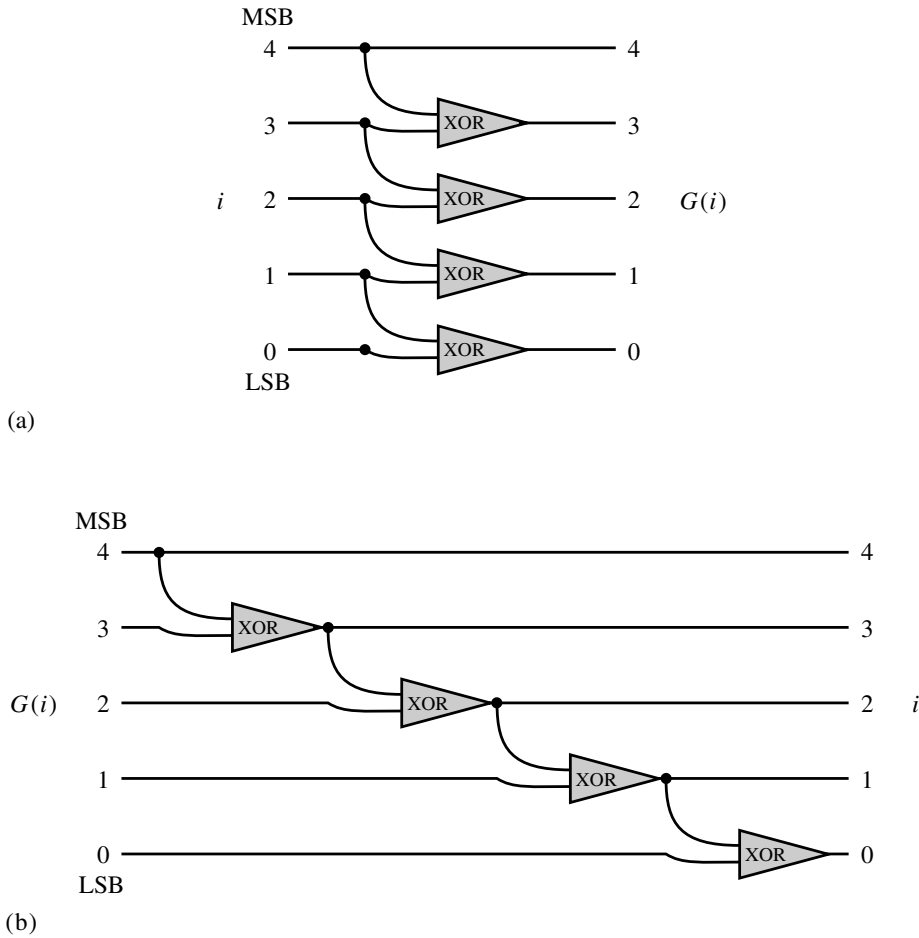


Figure 22.3.1. Single-bit operations for calculating the Gray code $G(i)$ from i (a), or the inverse (b). LSB and MSB indicate the least and most significant bits, respectively. XOR denotes exclusive-or.

```
struct Gray {
```

Methods for the Gray code and its inverse.

```
    UInt gray(const UInt n) {return n ^ (n >> 1);}
```

Return the Gray code of an integer n . This is the easy direction!

```
    UInt invgray(const UInt n) {
```

Return the inverse of the Gray code.

```
        Int ish=1;
```

```
        UInt ans=n, idiv;
```

```
        for (;;) {
```

ans ^= (idiv=ans >> ish); In hierarchical stages, starting with a one-bit right-shift, cause each bit to be XORed with all more significant bits.

```
        if (idiv <= 1 || ish == 16) return ans;
```

```
        ish <<= 1;
```

Double the amount of shift on the next cycle.

```
    }
```

```
};
```

igray.h

In numerical work, Gray codes can be useful when you need to do some task